

TAREA 1 - UD 2

Jose Cruces Aparicio 2º DAM

ODBC y JDBC

Resuelve

Como profesional del sector, sabes la importancia que tiene comprender y aplicar los conceptos de ODBC y JDBC, ya que estas son las tecnologías utilizadas en las aplicaciones para interactuar con los sistemas de gestión de bases de datos. Conocer las ventajas de utilizar sistemas de gestión de bases de datos independientes frente a embebidos y ser capaz de mostrar cómo pueden implementarse en aplicaciones es fundamental.



- 1. Investigar e indicar qué es ODBC y qué diferencias, ventajas y desventajas existen con respecto a JDBC.**

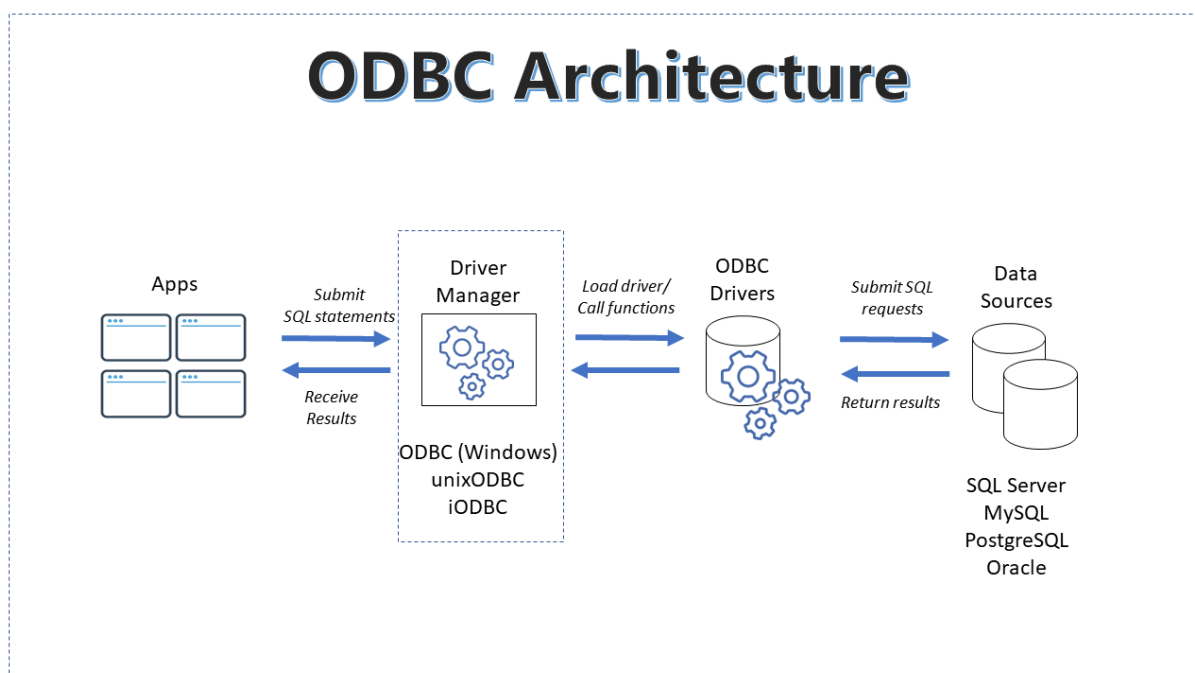
ODBC (Open Database Connectivity) es una interfaz estándar desarrollada para permitir que aplicaciones escritas en distintos lenguajes de programación accedan a sistemas de gestión de bases de datos de manera independiente del fabricante. ODBC actúa como una capa intermedia entre la aplicación y el SGBD, permitiendo ejecutar sentencias SQL sin necesidad de conocer los detalles específicos del motor de base de datos.

El funcionamiento de ODBC se basa en el uso de drivers. La aplicación realiza llamadas a la API de ODBC y el driver correspondiente traduce esas llamadas al lenguaje nativo del SGBD. Esto permite cambiar de base de datos sin modificar el código de la aplicación, siempre que exista un driver compatible.

JDBC (Java Database Connectivity) es la tecnología equivalente a ODBC en el entorno Java. Proporciona un conjunto de clases e interfaces que permiten a las aplicaciones Java conectarse a bases de datos relacionales, ejecutar consultas SQL y gestionar resultados. JDBC forma parte del estándar de Java, lo que garantiza su integración total con el lenguaje y su portabilidad entre plataformas.

La principal diferencia entre ODBC y JDBC radica en el entorno de uso. ODBC es independiente del lenguaje, pero depende del sistema operativo y de los drivers instalados. JDBC, al estar escrito completamente en Java, es multiplataforma y no requiere componentes externos más allá del propio driver JDBC.

Entre las ventajas de ODBC se encuentra su amplia compatibilidad con numerosos lenguajes y bases de datos. Sin embargo, su uso puede implicar una mayor sobrecarga y menor rendimiento debido a la capa adicional de abstracción. JDBC, por su parte, ofrece un mejor rendimiento en aplicaciones Java, una mayor seguridad y una integración directa con la gestión de excepciones y transacciones del lenguaje, aunque su uso queda limitado exclusivamente a Java.



2. Investigar e indicar las ventajas del uso de SGBD independientes frente a los SGBD embebidos. Muestra una captura de un programa que realices y que esté funcionando conectado a un SGBD independiente y a un SGBD embebido.

Los sistemas de gestión de bases de datos independientes funcionan como servicios externos a la aplicación, normalmente en arquitectura cliente-servidor. Ejemplos habituales son MySQL, MariaDB, PostgreSQL u Oracle. Estos sistemas están diseñados para entornos profesionales y multiusuario.

Una de sus principales ventajas es la capacidad de gestionar múltiples conexiones simultáneas, lo que los hace adecuados para aplicaciones empresariales y sistemas distribuidos. Además, ofrecen mecanismos avanzados de seguridad, como autenticación de usuarios, roles, permisos y cifrado de datos. También permiten realizar copias de seguridad, restauraciones y auditorías de forma centralizada.

Otra ventaja importante es la escalabilidad, ya que permiten crecer en número de usuarios y volumen de datos sin necesidad de cambiar de tecnología. La separación entre la aplicación y la base de datos facilita el mantenimiento, la actualización y la reutilización del sistema.

Por otro lado, los SGBD embebidos se integran directamente dentro de la aplicación y no requieren un servidor independiente. Ejemplos de este tipo son SQLite, H2 o Apache Derby. Suelen almacenar la base de datos en un único archivo local.

Las principales ventajas de los SGBD embebidos son su simplicidad y ligereza. No requieren instalación ni configuración compleja y ofrecen un buen rendimiento en aplicaciones de un solo usuario. Además, son altamente portables y resultan ideales para aplicaciones de escritorio, móviles, pruebas o prototipos.

En definitiva, los SGBD independientes son la mejor opción para aplicaciones grandes, multiusuario y críticas, mientras que los SGBD embebidos son más adecuados para aplicaciones sencillas, locales o de bajo consumo de recursos.



Código y capturas de dos proyectos para ver su funcionamiento y diferencias :

Proyecto 1. - MySQL + JDBC. - SGBD independiente

A continuación se muestra el código Java utilizado para establecer la conexión con la base de datos MySQL mediante JDBC, ejecutar una consulta y mostrar los resultados por consola.

```
import java.sql.*;

public class Acceso {
    private Connection conexion;
    private Statement st;
    private ResultSet rs;

    public Acceso() {
    }

    public void conectar() {
        try {
            // Cargar el driver moderno de MySQL
            Class.forName("com.mysql.cj.jdbc.Driver");

            // Datos de conexión
            String url = "jdbc:mysql://localhost:3306/aad?useSSL=false&serverTimezone=UTC";
            String usuario = "root";
            String contraseña = "tenerife";

            // Crear conexión
            conexion = DriverManager.getConnection(url, usuario, contraseña);
            st = conexion.createStatement();

            System.out.println("\n - Ya estamos conectados a la BD - \n");

        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    public void realizarConsulta(String consulta) {
        try {
            rs = st.executeQuery(consulta);

            // 🔍 Obtener metadatos del resultado
            ResultSetMetaData rsMetaData = rs.getMetaData();

            // Número de columnas
            int numColumnas = rsMetaData.getColumnCount();
        }
    }
}
```

```
System.out.println("Número de columnas: " + numColumnas);

// Nombre de la tabla (de la primera columna)
String nombreTabla = rsMetaData.getTableName(1);
System.out.println("Nombre de la tabla: " + nombreTabla);

// Nombre del esquema o catálogo
String catalogo = rsMetaData.getCatalogName(1);
System.out.println("Nombre del esquema o catálogo: " + catalogo);


System.out.println("\nDatos de la tabla alumnos con columnas:");
System.out.println("-----");

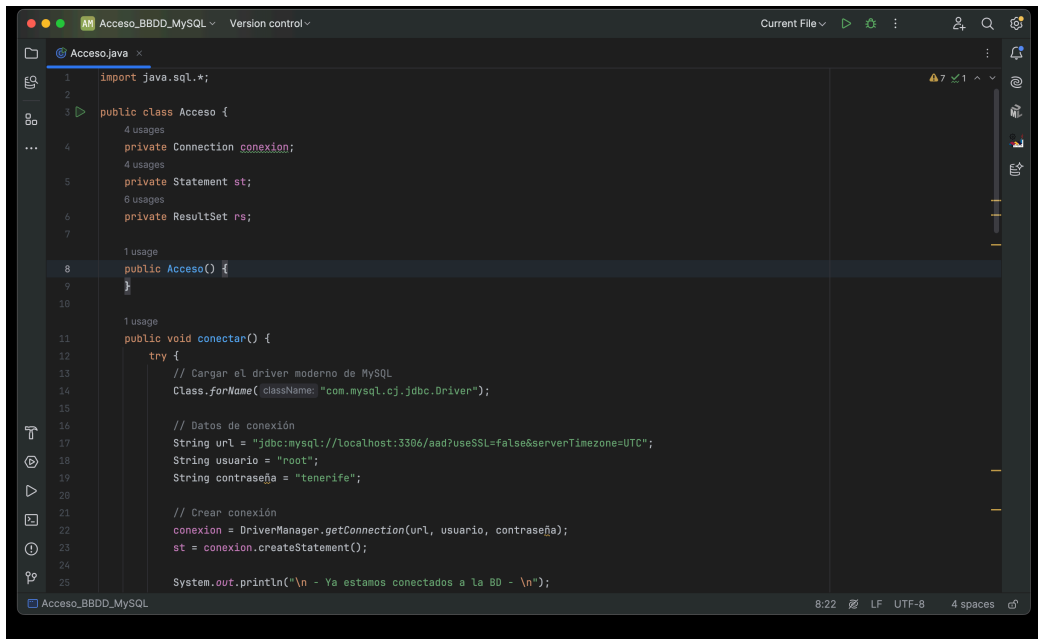
// Mostrar nombres de las columnas
for (int i = 1; i <= numColumnas; i++) {
    System.out.print(rsMetaData.getColumnName(i));
    if (i < numColumnas) System.out.print(" | "); // separador entre nombres
}
System.out.println(); // salto de línea final


// Mostrar los datos de la consulta
while (rs.next()) {
    for (int i = 1; i <= numColumnas; i++) {
        System.out.print(" " + rs.getString(i) + "\t");
    }
    System.out.println();
}

} catch (Exception e) {
    e.printStackTrace();
} finally {
    try {
        if (rs != null) rs.close();
        if (st != null) st.close();
        if (conexion != null) conexion.close();
        System.out.println("\n - Conexión cerrada -");
    } catch (SQLException e) {
        e.printStackTrace();
    }
}

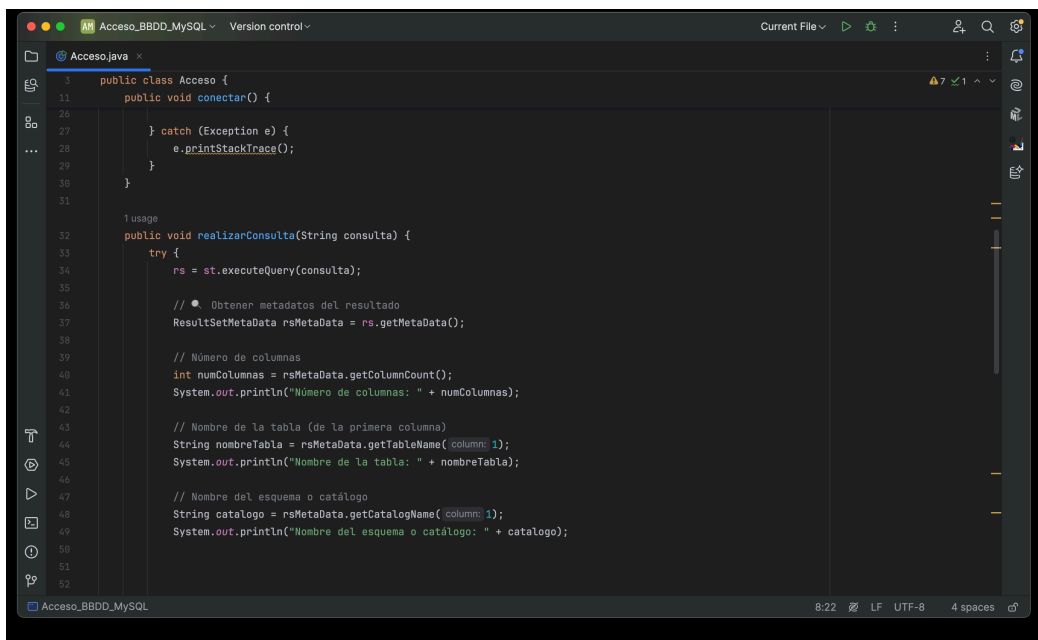
}

public static void main(String[] args) {
    Acceso acceso = new Acceso();
    acceso.conectar();
    acceso.realizarConsulta("SELECT * FROM Alumnos;");
}
}
```



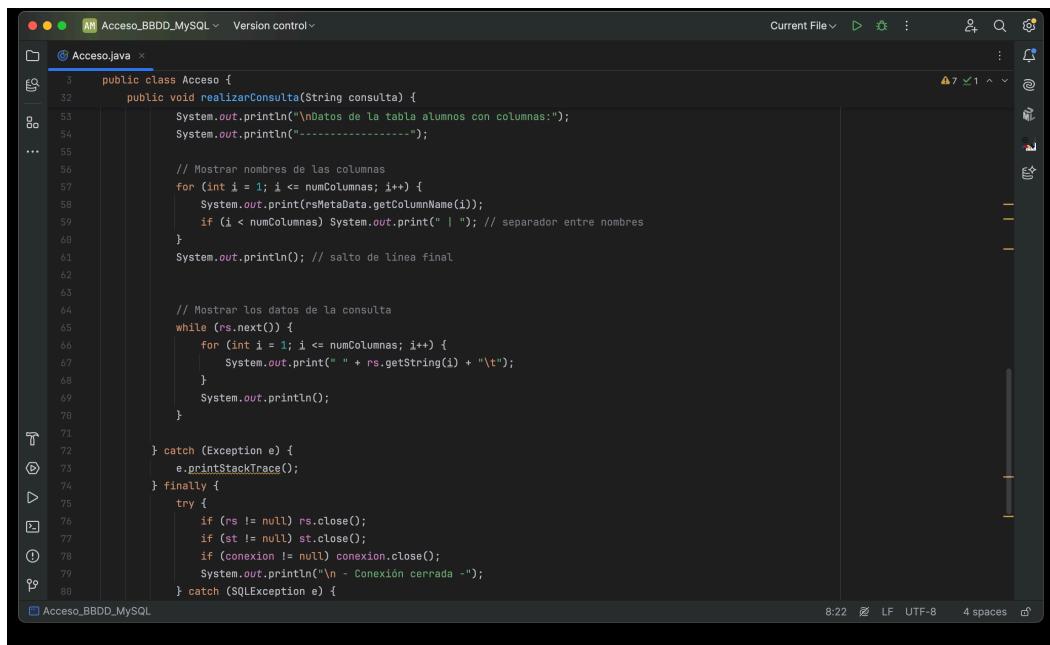
The screenshot shows an IDE window titled 'Acceso_BBDD_MySQL' with a file named 'Acceso.java'. The code defines a class 'Acceso' with private attributes 'conexion', 'st', and 'rs'. The 'conectar()' method is highlighted, showing a try-catch block that configures the MySQL connection URL, username, and password, and then creates the connection and statement objects. The status bar at the bottom indicates the file is in UTF-8 encoding with 4 spaces.

```
1 import java.sql.*;
2
3 public class Acceso {
4     private Connection conexion;
5     private Statement st;
6     private ResultSet rs;
7
8     public Acceso() {}
9
10
11     public void conectar() {
12         try {
13             // Cargar el driver moderno de MySQL
14             Class.forName("com.mysql.cj.jdbc.Driver");
15
16             // Datos de conexión
17             String url = "jdbc:mysql://localhost:3306/aad?useSSL=false&serverTimezone=UTC";
18             String usuario = "root";
19             String contraseña = "tenerife";
20
21             // Crear conexión
22             conexion = DriverManager.getConnection(url, usuario, contraseña);
23             st = conexion.createStatement();
24
25             System.out.println("\n - Ya estamos conectados a la BD - \n");
```



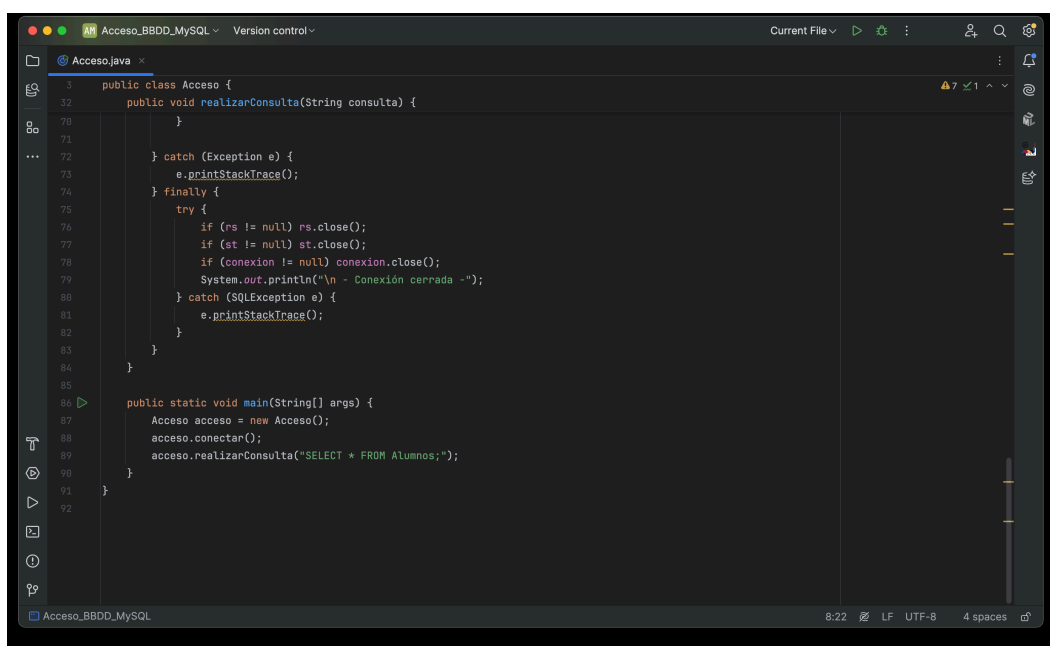
The screenshot shows the same IDE window, but now the 'realizarConsulta()' method is visible. This method takes a query string as input, executes it using the 'st' statement object, and then retrieves and prints metadata about the result set, including the number of columns, the table name, and the schema name.

```
32 public void realizarConsulta(String consulta) {
33     try {
34         rs = st.executeQuery(consulta);
35
36         // Obtener metadatos del resultado
37         ResultSetMetaData rsMetaData = rs.getMetaData();
38
39         // Número de columnas
40         int numColumnas = rsMetaData.getColumnCount();
41         System.out.println("Número de columnas: " + numColumnas);
42
43         // Nombre de la tabla (de la primera columna)
44         String nombreTabla = rsMetaData.getTableName(1);
45         System.out.println("Nombre de la tabla: " + nombreTabla);
46
47         // Nombre del esquema o catálogo
48         String catalogo = rsMetaData.getCatalogName(1);
49         System.out.println("Nombre del esquema o catálogo: " + catalogo);
50
51
52 }
```



This screenshot shows the 'realizarConsulta' method in the 'Acceso.java' file. The method is designed to execute a SQL query and display its results. It starts by printing the query string, then iterates through the column names of the result set. After displaying the column names, it enters a loop to print each row of data. The method includes a try-catch block to handle any database-related exceptions and a finally block to ensure that the result set, statement, and database connection are properly closed.

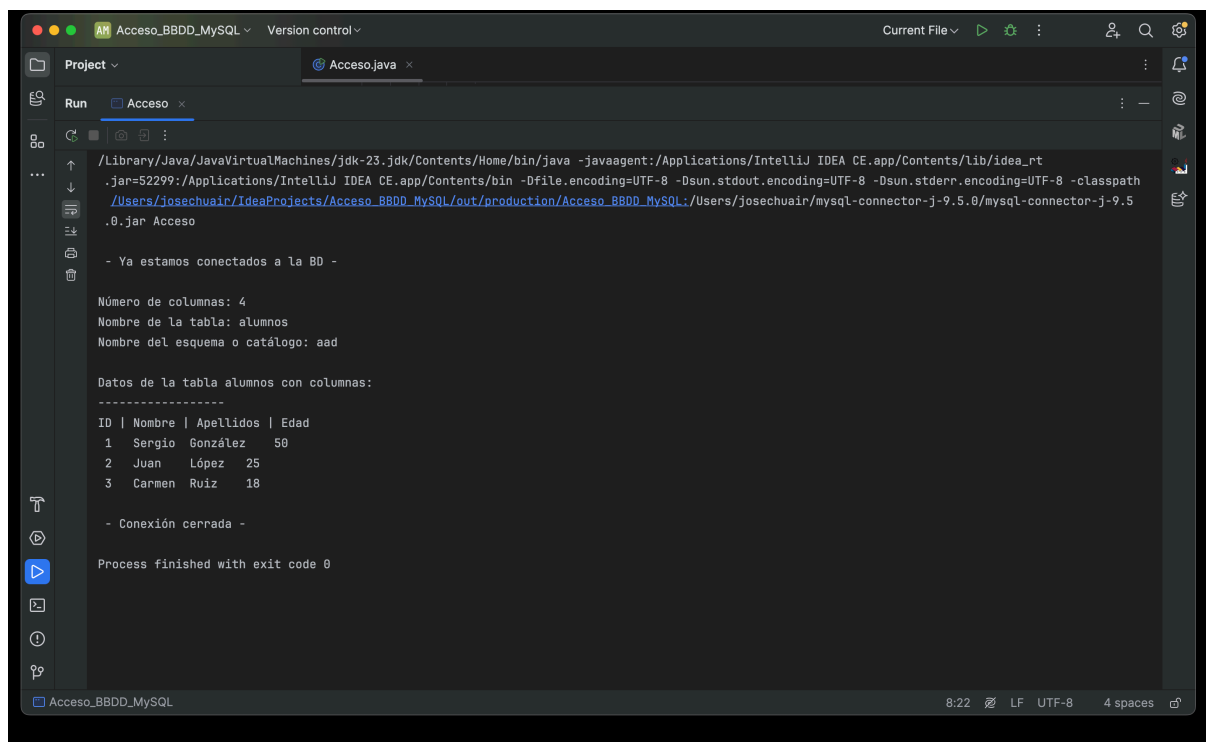
```
3 public class Acceso {
32 public void realizarConsulta(String consulta) {
53     System.out.println("\nDatos de la tabla alumnos con columnas:");
54     System.out.println("-----");
55
56     // Mostrar nombres de las columnas
57     for (int i = 1; i <= numColumnas; i++) {
58         System.out.print(rsMetaData.getColumnNames(i));
59         if (i < numColumnas) System.out.print(" | "); // separador entre nombres
60     }
61     System.out.println(); // salto de línea final
62
63
64     // Mostrar los datos de la consulta
65     while (rs.next()) {
66         for (int i = 1; i <= numColumnas; i++) {
67             System.out.print(" " + rs.getString(i) + "\t");
68         }
69         System.out.println();
70     }
71
72 } catch (Exception e) {
73     e.printStackTrace();
74 } finally {
75     try {
76         if (rs != null) rs.close();
77         if (st != null) st.close();
78         if (conexion != null) conexion.close();
79         System.out.println("\n - Conexión cerrada -");
80     } catch (SQLException e) {
81         e.printStackTrace();
82     }
83 }
84 }
```



This screenshot shows the 'main' method in the 'Acceso.java' file. It creates an instance of the 'Acceso' class, connects to the database, and then calls the 'realizarConsulta' method with a SQL query to select all data from the 'Alumnos' table.

```
86 public static void main(String[] args) {
87     Acceso acceso = new Acceso();
88     acceso.conectar();
89     acceso.realizarConsulta("SELECT * FROM Alumnos;");
90 }
91
92 }
```

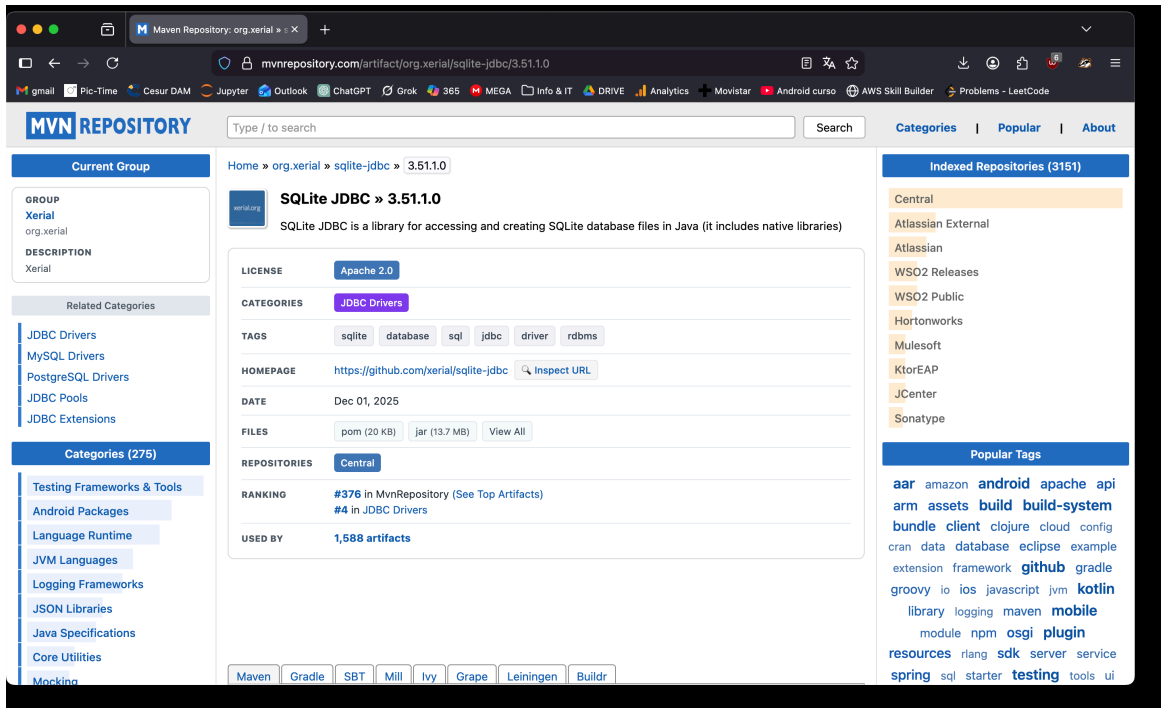
Arrancamos el servidor con XAMPP y ejecutamos la aplicación en IntelliJ para ver el resultado :



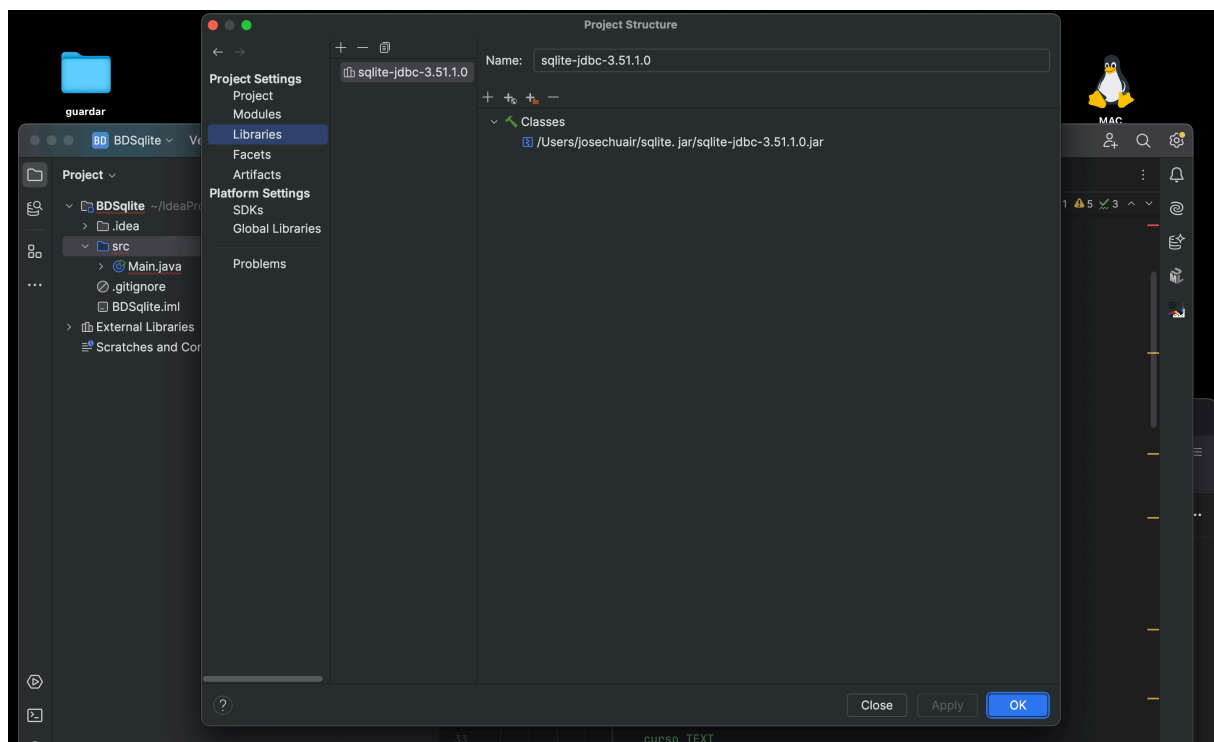
```
Accesso_BBDD_MySQL  Version control
Current File  Accesso.java
Run  Acceso
/Library/Java/JavaVirtualMachines/jdk-23.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE.app/Contents/Lib/idea_rt.jar=52299:/Applications/IntelliJ IDEA CE.app/Contents/bin -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath /Users/josechuair/IdeaProjects/Accesso_BBDD_MySQL/out/production/Accesso_BBDD_MySQL:/Users/josechuair/mysql-connector-j-9.5.0/mysql-connector-j-9.5.0.jar Acceso
- Ya estamos conectados a la BD -
Número de columnas: 4
Nombre de la tabla: alumnos
Nombre del esquema o catálogo: aad
Datos de la tabla alumnos con columnas:
-----
ID | Nombre | Apellidos | Edad
1 | Sergio | González | 50
2 | Juan | López | 25
3 | Carmen | Ruiz | 18
- Conexión cerrada -
Process finished with exit code 0
```


Proyecto 2 conexión SQLITE + JDBC. SGBD embebido

Buscamos el driver SQLite ultima versión en maven repository , lo descargamos y lo añadimos al proyecto en las librerías que tiene que utilizar , generamos el código para que accedamos a la base de datos y muestro las capturas de pantalla del proyecto :



Cargamos en el proyecto el jar en Project Structure :



A continuación se muestra el código Java utilizado para establecer la conexión con la base de datos SQLite mediante JDBC, ejecutar una consulta y mostrar los resultados por consola.

```
import java.sql.*;

public class AccesoSQLite {

    private Connection conexion;
    private Statement st;
    private ResultSet rs;

    public void conectar() {
        try {
            // Cargar driver SQLite
            Class.forName("org.sqlite.JDBC");

            // Base de datos embebida (archivo local)
            String url = "jdbc:sqlite:alumnos.db";

            conexion = DriverManager.getConnection(url);
            st = conexion.createStatement();

            System.out.println("\n - Conectado a SQLite (SGBD embebido) - \n");

        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    public void crearTabla() {
        try {
            String sql = ""
                CREATE TABLE IF NOT EXISTS Alumnos (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    nombre TEXT,
                    curso TEXT
                );
            """;

            st.execute(sql);
            System.out.println("Tabla creada o ya existente.");

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

```
public void insertarDatos() {
    try {
        String sql = "INSERT INTO Alumnos (nombre, curso) VALUES ('Juan', '2º DAM')";
        st.executeUpdate(sql);
        System.out.println("Datos insertados.");

    } catch (SQLException e) {
        e.printStackTrace();
    }
}

public void realizarConsulta() {
    try {
        rs = st.executeQuery("SELECT * FROM Alumnos");

        System.out.println("\nDatos de la tabla Alumnos:");
        System.out.println("-----");

        while (rs.next()) {
            System.out.println(
                rs.getInt("id") + " - " +
                rs.getString("nombre") + " - " +
                rs.getString("curso")
            );
        }

    } catch (SQLException e) {
        e.printStackTrace();
    }
}

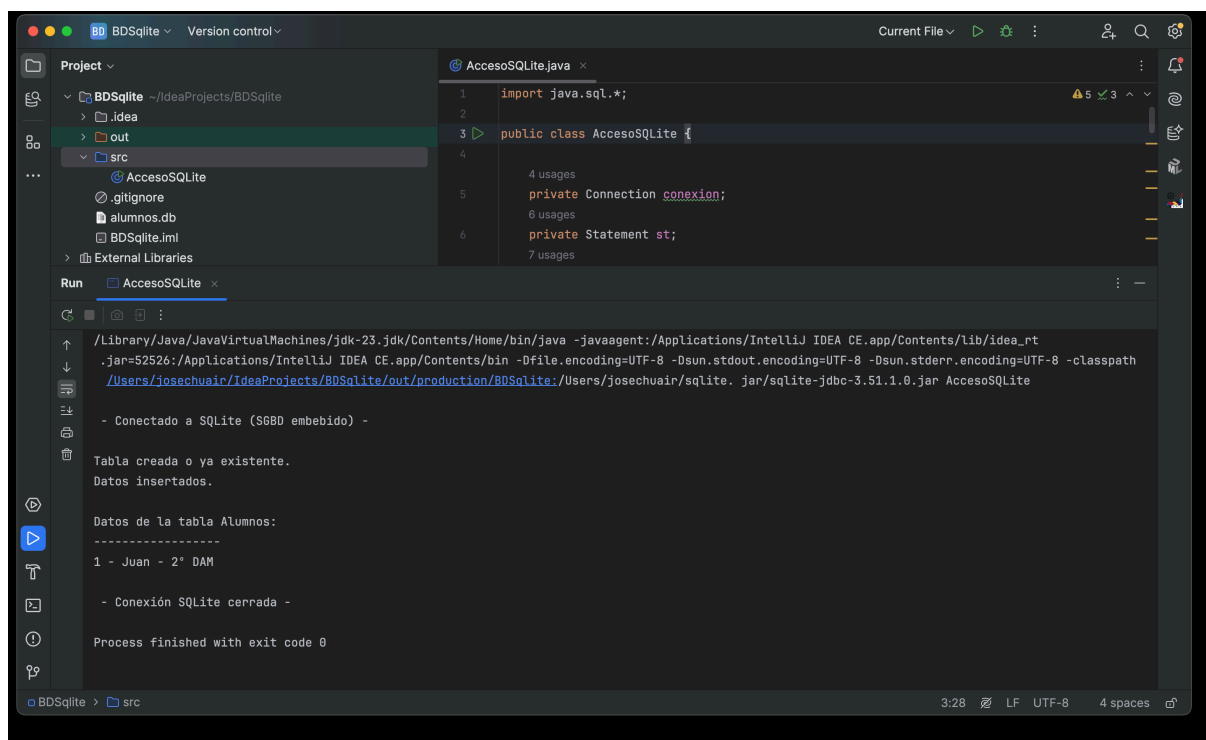
public void cerrarConexion() {
    try {
        if (rs != null) rs.close();
        if (st != null) st.close();
        if (conexion != null) conexion.close();
        System.out.println("\n - Conexión SQLite cerrada -");

    } catch (SQLException e) {
        e.printStackTrace();
    }
}

public static void main(String[] args) {
    AccesoSQLite acceso = new AccesoSQLite();
}
```

```
    acceso.conectar();  
    acceso.crearTabla();  
    acceso.insertarDatos();  
    acceso.realizarConsulta();  
    acceso.cerrarConexion();  
}  
}
```

Mostramos la ejecución del programa con una captura de pantalla , basico para conectar a la base de datos y crear una tabla sencilla pero suficiente para mostrar la conexión correcta. Incluso podemos ver como creo el archivo alumnos.db en la carpeta



3. Hacer una tabla indicando cada uno de los drivers JDBC estudiados con sus ventajas y desventajas. Indicar, además, en qué casos es más recomendable el uso de cada uno de los tipos.

Tipo de driver	Descripción	Ventajas	Desventajas	Uso recomendado
Tipo 1 (JDBC-ODBC Bridge)	Utiliza ODBC como intermediario entre Java y la base de datos.	Fácil de usar, útil para aprendizaje.	Bajo rendimiento, dependiente de ODBC, actualmente obsoleto.	Solo para prácticas y aprendizaje.
Tipo 2 (API nativo)	Combina código Java con librerías nativas del SGBD.	Mejor rendimiento que el Tipo 1.	Dependiente del sistema operativo y librerías externas.	Entornos muy controlados.
Tipo 3 (Protocolo de red)	Usa un servidor intermedio para comunicarse con la base de datos.	Multiplataforma y flexible.	Mayor complejidad y necesidad de servidor adicional.	Sistemas distribuidos.
Tipo 4 (Protocolo nativo)	Se conecta directamente al SGBD usando su protocolo nativo.	Mejor rendimiento, todo Java, multiplataforma.	Dependiente del SGBD concreto.	Aplicaciones reales y profesionales (el más usado).

Los drivers JDBC se clasifican en cuatro tipos según su arquitectura. El driver Tipo 1, conocido como JDBC-ODBC Bridge, fue utilizado principalmente con fines educativos, pero actualmente está obsoleto debido a su bajo rendimiento y dependencia de ODBC. El Tipo 2 mejora el rendimiento al utilizar librerías nativas, pero pierde portabilidad.

El driver Tipo 3 introduce un servidor intermedio, lo que lo hace adecuado para arquitecturas distribuidas, aunque añade complejidad. Finalmente, el driver Tipo 4 es el más utilizado en la actualidad, ya que se comunica directamente con el SGBD utilizando su protocolo nativo y está escrito completamente en Java, ofreciendo el mejor rendimiento y portabilidad.

Bibliografía. :

UD 2 Acceso de datos

<https://mundobytes.com/para-que-sirve-odbc/>

<https://docs.oracle.com/javase/tutorial/jdbc/>

<https://www.sqlite.org/docs.html>

<https://dev.mysql.com/doc/connector-j/en/>